

论数据架构的设计与应用

⚠ 关于本文中的数字 本文的量化数据（数据量、查询延迟、作业耗时等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字**（表数量、接口数、配置值）已从项目源码核实，可直接使用；**业务规模类数字**（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据。文末附有「待替换数字清单」，便于逐项核对。

试题

论数据架构的设计与应用

数据架构是描述组织数据资产的结构、关系与管理规则的蓝图，它决定了数据如何被采集、存储、处理、共享和使用。良好的数据架构能够支撑业务的快速变化，保障数据的一致性与可用性，并为数据分析与决策提供基础。数据架构设计需要综合考虑数据特征、访问模式、一致性要求、扩展性需求以及成本约束等多方面因素。

请围绕“数据架构的设计与应用”论题，依次从以下三个方面进行论述：

- 简要叙述你参与设计和开发的系统的数据架构，以及你所担任的工作；
- 详细论述你在该系统中所采用的存储选型依据、数据模型设计方法以及一致性保障措施；
- 分析你在数据架构设计与实施中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了该平台数据架构的设计与应用。该平台承载预约管理、客户消息、通知发送与主数据联动等业务，其数据形态差异极大，既有要求事务与关联查询的强一致交易数据，也有字段频繁演进的半结构化数据，还有体积较大的文件实体。我在项目中担任系统架构师，负责数据模型设计、存储选型与数据一致性方案设计。针对单一存储难以同时兼顾强一致与弹性扩展、异构存储之间的一致性难以保障、批量作业与在线业务争抢数据资源三大挑战，我采用混合存储策略，以关系型数据库承载强一致交易，以云数据库承载高弹性半结构化数据，以对象存储承载文件实体，并通过事件驱动链路达成最终一致；同时为批量作业配置独立的读写数据源实现读写分离。系统上线后运行稳定，数据查询延迟与批量作业耗时均明显改善。

正文

一、项目背景与数据挑战

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔。我在项目中担任系统架构师，负责数据模型设计、存储选型与数据一致性方案设计。

在项目启动阶段，我首先对平台的数据进行了分类梳理，最终归纳为四类，这四类数据的特征差异构成了数据架构设计的全部依据。

第一类是强一致交易数据，包括预约、申请人、旅行者、费用明细等。这类数据之间存在复杂的关联关系，查询时往往需要多表联结，且写入必须保证事务性。第二类是字段频繁演进的半结构化数据，典型如旅行主信息——其字段随各国签证政策与集团业务规则频繁增删，若以关系表承载，每一次变更都需要执行表结构变更，代价高且风险大。第三类是文件实体，包括护照扫描件、各类文档与亲权同意书，特点是体积大、访问频率低、需要独立的对象存储能力。第四类是高读写小数据，如消息线程与流水号，读写频繁且要求低延迟。

这四类数据的存在，使“用一种存储解决所有问题”的思路从一开始就走不通：关系型数据库在强一致与关联查询上不可替代，但在字段灵活性与弹性扩展上存在明显短板；NoSQL 数据库在模式灵活与横向扩展上优势突出，却难以提供跨表事务；对象存储适合大文件，却不适合承载业务查询。此外，批量作业与在线业务访问同一数据源时，批量扫描会挤占在线业务的连接资源，这是我们在早期预演中就已发现的风险。

二、存储选型与数据模型设计

基于上述数据分类，我确立了混合存储的选型方案，选型依据是数据的一致性要求、访问模式与演进频率。

关系型数据库承载强一致交易。 预约、申请人、旅行者、费用等需要事务与复杂关联查询的数据，全部落在关系型数据库，共 79 张表，其中主数据表 26 张（m_ 前缀）、业务表 53 张（t_ 前缀）。主数据与业务数据在物理上分离组织，便于分别治理：主数据由外部主数据平台同步而来，业务侧只读；业务数据承载日常交易，读写频繁。

云数据库承载高弹性半结构化数据。 对于字段演进频繁、访问模式以单条读写为主的数据，我选择云数据库，共设计了 5 张表，分别承载旅行信息、文档元数据、亲权同意书、消息线程与流水号。

在此我重点说明**单表多实体的设计方法**。以旅行信息表为例，我采用“分区键 + 排序键 + 载荷字段”的模型：以旅行编号作为分区键，以带类型前缀的标识作为排序键（消息线程以 THR# 开头、消息以 MSG# 开头、文件以 FIL# 开头），实体内容以结构化文本形式存于载荷字段。这样设计带来三个好处：其一，一张表可以承载同一旅行下的多种数据类型，避免了表数量随业务类型线性膨胀；其二，同一旅行的全部数据落在同一分区，查询时可以通过一次分区查询取回，无需跨分区扫描，这是延迟可控的关键；其三，字段的增删只需调整载荷内容，完全不需要执行表结构变更，业务规则的快速迭代不再受阻于数据库层面。

对象存储承载文件实体。 护照、文档、亲权同意书等文件统一存放于对象存储，关系库与云数据库中只保留文件的引用信息。这一拆分使数据库的体积与备份恢复时长大幅下降。

流水号的分布式生成。 平台需要跨服务生成唯一且有序的业务编号，关系型数据库的自增机制无法满足分布式场景。为此我在云数据库中设计了一张专用的流水号表，以业务类型作为键、当前序号作为值，通过条件更新保证并发安全。

三、数据访问与一致性保障

数据访问层的组织。 关系型数据库侧统一采用 MyBatis 作为持久层框架，并通过统一版本管理插件锁定其版本，避免因版本漂移导致行为不一致。实体按主数据与业务数据分离组织，数据访问接口按业务域归类。云数据库侧通过增强客户端提供对象映射能力，通用操作封装在仓储基类中，实体属性名集中定义在常量类中，避免魔法字符串散落各处。为控制全表扫描的风险，我把扫描上限显式设置为 10000 条，为可能出现的误用设置硬边界——这个上限远高于任何正常的单次查询所需，因此它不影响正常业务，却能在代码误用导致全表扫描时及时失效而非拖垮数据库。

并发写入的控制。 云数据库侧不支持关系库那样的行锁机制，因此我采用乐观锁配合重试的策略：为需要并发控制的实体引入版本属性，更新时校验版本，冲突则以最大 4 次尝试、1 秒间隔重试。这一方案在预约信息更新等场景下运行良好。

跨存储的最终一致性。混合存储必然带来一致性问题。写入主链路落在云数据库，而报表与既有系统仍在关系库中查询，两者需要同步。我的方案是把同步设计为事件驱动：以云数据库的数据流捕获变更事件，以消息队列承接业务消息，驱动十四个按业务域划分的函数完成同步。主链路只写云数据库即返回，同步异步进行，达成秒级的最终一致性。需要说明的是，这一设计是有意为之的权衡——预约写入等强一致场景仍走关系库事务，不做异步化；只有报表与通知这类可以容忍秒级延迟的场景才走异步链路。

主数据的本地化。集团主数据由外部平台统一管理，若业务查询直连外部系统，则外部系统的任何抖动都会传导至业务操作。因此我设计了一个定时作业，从主数据接口拉取本国、城市、航空公司、货币等主数据，落地为本地只读副本，业务侧只查本地。数据新鲜度通过同步频率控制，这是以可用性优先的合理取舍。

四、实施中遇到的问题及解决措施

问题一：批量作业与在线业务争抢数据资源。平台的成本文件生成、外部预约导入、待办状态更新等批量作业需要从生产库大量读取并回写结果，而在线预约接口同时也在访问同一数据库。若共享连接池，夜间批处理峰值会直接推高在线接口的延迟。我的解决方案是为批处理配置独立的读写数据源：读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口，使读写请求在物理层面分离。同时，我把事务隔离级别设置为读已提交，减少长事务持有锁的时间；并把批量游标大小限制在 200 条，防止海量结果集一次性载入导致内存溢出——这是批处理最常见的故障模式。改造后，批量作业耗时下降约 30%，在线接口的响应时间不再受批量作业影响。

问题二：跨国部署下的时间一致性。平台服务日本本部，源数据库运行于日本标准时间，而容器运行环境为协调世界时，开发终端又是另一个时区。三套时区并存时，预约截止时间、邮件发送时刻等跨时区比较极易出错，后果是提前发送或漏发——对旅游业务而言这是不可接受的。我的处理原则是时间口径单一化：存储层统一使用协调世界时；展示层按业务方时区转换；转换动作集中到统一的格式化工具类，禁止在业务代码中散落直接取当前时间的调用。批处理侧更为彻底——执行基准日与取得范围作为作业参数显式传入，不依赖系统时钟，使同一参数的重跑结果完全可复现。

问题三：主数据与业务数据的一致性。初期曾出现主数据更新后业务侧仍在用旧值的情况。经排查，根源是同步作业失败后没有可观测的手段，运维人员无从知晓。我为此补充了两项措施：一是把同步作业的执行结果纳入监控，作业失败即触发告警；二是让同步作业的基准日期参数化，使补数可以精确指定日期范围重跑。改造后，主数据不一致的问题单数量下降约 90%。

五、总结与展望

平台上线至今已稳定运行约 8 个月。数据架构方面的成效主要体现在：云数据库侧的单条查询延迟稳定在 10 毫秒以内；批量作业耗时下降约 30%；数据存储成本因文件外置而下降约 25%；跨存储同步延迟稳定在秒级。

目前仍存在两点不足：一是混合存储引入了运维复杂度，多套存储的备份策略、容量规划与权限管理需要分别维护，对运维团队的要求较高；二是跨存储的一致性链路缺少统一的延迟监控，异常时的排查需要分别在多个系统中查看日志，路径偏长。后续计划引入统一的链路追踪与一致性校验作业，对关键数据定期执行对账。

通过本项目，我对数据架构设计形成了三点认识。第一，不存在“最好的存储”，只存在“与数据访问特征最匹配的存储”，选型的依据应当是数据本身的一致性要求、访问模式与演进频率，而不是技术的新旧。第二，服务边界必须与数据边界对齐——如果多个服务共享同一张表，那么无论代码如何拆分，数据层面依然是耦合的。第三，混合存储的代价必须被正视：一致性、运维复杂度与排查难度都会上升，架构师的责任是在采用之前就明确这些代价，并设计好补偿措施。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量	约 3 万笔	真实数量
关系库表数量	79 张（主数据 26 + 业务 53）	✓ 已核实
云数据库表数	5 张	✓ 已核实
单条查询延迟	10 毫秒以内	真实数据
批量游标 / 扫描上限	200 条 / 10000	✓ 已核实
批量作业耗时下降	约 30%	真实数据
存储成本下降	约 25%	真实数据
主数据问题单下降	约 90%	真实数据
稳定运行时长	约 8 个月	真实时长

本文字数

摘要约 330 字，正文约 2780 字，**合计约 3110 字**。

可复用要点

- **四类数据分类法**（强一致 / 半结构化 / 文件 / 高频小数据）是本文最有价值的方法论，可直接迁移到任何数据架构题目
- **单表多实体（分区键 + 排序键 + 载荷）** 是一个有技术深度的设计点，多数考生写不出来
- **多时区一致性** 是跨国项目的特色难点，几乎所有"数据架构""批处理""国际化"类题目都能用
- 若题目侧重"数据治理"，把主数据本地化与监控告警展开；若侧重"性能"，把读写分离展开